

# APML: Tutorial Session

Nan Li

n.li AT uu.nl

November 12th, 2024



Universiteit Utrecht

# Me

## Nan Li

- your Teaching Assistant; a first-year Ph.D. in NLP
- Office: BBG 5.05
- Email: n.li AT uu.nl
- Lab Sessions:
  - Tuesday 11:00-12:45
  - Thursday 15:15-17:00
  - Location: Dalton 7.27

# To-do List

- 1 Introduction
- 2 Teamwork with GitHub
- 3 Python: Jupyter Notebook
- 4 Assignment 1: Outlier Detection

- 1 Introduction
- 2 Teamwork with GitHub**
- 3 Python: Jupyter Notebook
- 4 Assignment 1: Outlier Detection

# Why GitHub

- **Version Control:** GitHub enables efficient version control, allowing teams to keep track of code changes over time, which is essential for managing projects.
- **Collaboration:** Team members can work on the same codebase simultaneously, contributing to different parts without overwriting each other's work.
- **Cloud Storage and Backup:** Projects are stored on GitHub's servers, making code accessible from anywhere and acting as a backup.

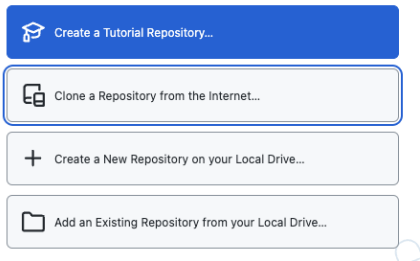
# Git vs. GitHub

- **Git:** A local version control tool that tracks changes on your computer. It's installed locally and allows you to commit, branch, and merge.
- **GitHub:** A cloud-based platform that hosts Git repositories online, making it easier for teams to collaborate remotely.

# Basic: GitHub Desktop

- **Overview:** GitHub Desktop is a graphical interface for Git. It simplifies Git tasks without requiring command-line input, ideal for beginners.
- **Setup:** Download and install GitHub Desktop from the official website. Sign in to your GitHub account.
- **Create a Repository:**
  - Open GitHub Desktop, click `File > New Repository`.
  - Choose a name and local path, then click `Create`.
- **Clone a Repository:**
  - Click `File > Clone Repository`.
  - Enter the URL of the repository and select a save location.

# Screenshots: A New Repository



### Create a New Repository

×

Name

Description

Local Path

Choose...

☐ Initialize this repository with a README

Git Ignore

None

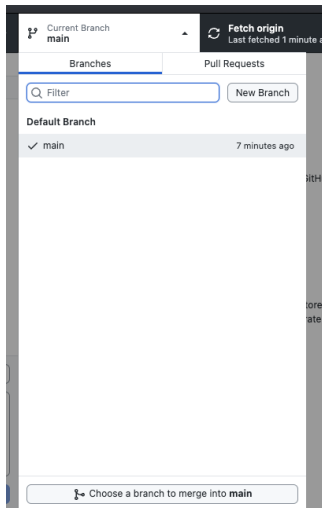
License

None

CancelCreate Repository



# Basic: Branch



- **Isolated Workspaces** A branch in Git acts as an isolated workspace where you can make changes without affecting the main codebase or other branches. This allows developers to experiment or work on new features independently.
- **Version Control and Merging** Once the work in a branch is complete, it can be reviewed and merged into the main codebase. This helps track changes over time and integrate updates in a controlled way, reducing conflicts and improving collaboration.

# Basic: Push and Pull

- **Push Commits:** Save and upload changes to the GitHub repository.
  - Make changes, select the modified files, and write a commit message.
  - Click `Commit to main`, then `Push origin` to upload changes.
- **Pull Updates:** Download the latest changes from the repository.
  - Click `Fetch origin` to see new updates.
  - Click `Pull origin` to sync your local repo with the latest version.
- **Create Pull Requests:** Suggest changes and request review from teammates.
  - Go to the repository page on GitHub, open the `Pull Requests` tab.
  - Click `New pull request`, add a title, description, and request a review.

# Screenshots: Make a commit

The screenshot displays the GitHub Desktop application window. At the top, the 'Current Repository' is 'desktop-tutorial', the 'Current Branch' is 'main', and the 'Fetch origin' button shows it was last fetched 9 minutes ago. The 'Changes' tab is active, showing '1 changed file' and 'README.md'. The diff view for README.md shows changes on lines 3, 4, 5, 6, and 7. Line 6 has a green highlight and a '+' icon, and line 7 also has a '+' icon. The right sidebar contains a 'Get started' section with a list of steps: 1. Install a text editor (checked), 2. Create a branch, 3. Edit a file, 4. Make a commit (highlighted with a blue arrow), 5. Publish to GitHub, and 6. Open a pull request. Below the diff view, there is a 'Test' input field with the text 'add one line "an edit"', a commit message input field, and a 'Commit to main' button.

Current Repository: desktop-tutorial

Current Branch: main

Fetch origin: Last fetched 9 minutes ago

Changes: 1 changed file

History

README.md

@@ -3,3 +3,5 @@

3 3 This is your README. READMEs are where you can communicate what your project is and how to

4 4

5 5 Write your name on line 6, save it, and then head back to GitHub Desktop.

6 +

7 + An edit.

Get started

- 1 Install a text editor
- 2 Create a branch
- 3 Edit a file
- 4 Make a commit
- 5 Publish to GitHub
- 6 Open a pull request

A commit allows you to save sets of changes. In the "summary" field in the bottom left, write a short message that describes the changes you made. When you're done, click the blue Commit button to finish.

Test

add one line "an edit"

Commit to main

# Advanced: Git Command Line

- **Setting Up Git** Install Git, configure username and email using:
  - `git config --global user.name "Your Name"`
  - `git config --global user.email "you@example.com"`
- **Basic Commands:**
  - `git init`: Initializes a new Git repository.
  - `git add <filename>`: Stages a file for committing.
  - `git commit -m "message"`: Commits staged changes.
  - `git push`: Uploads commits to GitHub.
  - `git pull`: Downloads and merges changes from GitHub.

- 1 Introduction
- 2 Teamwork with GitHub
- 3 Python: Jupyter Notebook**
- 4 Assignment 1: Outlier Detection

# Data Exploration

- Let's open APML tutorial 1 - `data_exploration.ipynb` ...

- 1 Introduction
- 2 Teamwork with GitHub
- 3 Python: Jupyter Notebook
- 4 Assignment 1: Outlier Detection**

# Description

- Let's open Assignment 1.ipynb ...